School of Computer Science: assessment brief

Module title	Parallel Computation
Module code	COMP3221
Assignment title	Coursework 3
Assignment type and description	OpenCL Programming Assignment
Rationale	Implementation of a general purpose GPU program in OpenCL. Correctly implement a GPU kernel to perform a common numerical task.
Guidance	See overpage
Weighting	15%
Submission deadline	2pm, Tuesday 5th May
Submission method	Gradescope
Feedback provision	Marks and comments returned *via* Gradescope
Learning outcomes assessed	Apply parallel design paradigms to serial algorithms. Evaluate and select appropriate parallel solutions for real world problems.
Module lead	David Head

1. **Assignment guidance**

   This coursework specification is for school Unix machines only, including the remote access `feng-linux.leeds.ac.uk/gpu`. We cannot guarantee it will work on any other environment.

   For this coursework you are required to design an OpenCL application that transposes a matrix in parallel on a GPU. Transposition is a common numerical operation that arises in many applications. For a 2×4 matrix — a matrix with 2 rows and 4 columns — the transpose operation looks like this:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{pmatrix} \quad \rightarrow \quad \begin{pmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{pmatrix}$$

   Observe that what were once rows of the matrix are now columns, and the matrix itself has changed shape, from 2×4 to 4×2, although the total size is the same: $2 \times 4 = 4 \times 2 = 8$.

   Although matrices are often represented as 2-dimensional arrays, for this coursework (as with the previous one) they are stored in 1-dimensional arrays – this makes it easier to copy data to and from the device. For an `nRows` by `nCols` matrix, the total array size is `nRows*nCols`, and the `i`'th row and `j`'th column has index `i*nCols+j` in the 1-dimensional array. This needs be mapped to the index `j*nRows+i` in the transposed matrix.

   You will need to material up to and including Lecture 16 for this assignment.

2. **Assessment tasks**

   The following files are available on Minerva.

`cwk3.c`	:	The starting point for your solution.
`ckw3.cl`	:	The kernel code. Currently this file only contains a comment.
`helper_cwk.h`	:	Includes the same helper routines as used in the lecture examples, plus some specific routines for this coursework.
`makefile`	:	A simple makefile that selects between `nvcc -lOpenCL` for Linux systems (*e.g.* school machines), and `gcc -framework OpenCL` for Macs. Usage optional. On School machines you still need to load the CUDA module first; see Lecture 14.

   The code `cwk3.c` initialises a matrix called `hostMatrix` with random values, with the size of the matrix given by command line arguments. After building the code as per the instructions in lectures, execute as *e.g.*

```
./cwk3 16 8
```

   which will initialise a matrix with 16 rows and 8 columns, and fill it with random values in the range 0 to 1. It will then display the matrix, followed by the transposed version that is currently incorrect.

You need to add an OpenCL kernel and calling code to perform the transpose correctly. Your solution should work with matrices of size `nRows` by `nCols`, and should be as efficient as possible using the material covered in Lectures 14 to 16 inclusive.

Both `nRows` and `nCols` can be assumed to be powers of 2; the routine `getCmdLineArgs` in `helper_cwk.h` checks for this. You can assume that the matrix is small enough that memory allocation on the device is always possible.

3. **General guidance and study support**

   If you have any queries about this coursework, visit the Teams page for this module. If your query is not resolved by previous answers, post a new message. Support will also be available during the timetabled lab sessions.

   Note that it is perfectly possible that the best solution to a task is not very sophisticated, given the material covered in lectures. You should identify such cases and implement what may be a simple solution.

4. **Assessment criteria and marking process**

   Your code will be checked on a School machine that matches the GPU-enabled remote machine `feng-linux.leeds.ac.uk/gpu`. Staff will then inspect your code and allocate the marks as per the mark scheme (see below).

   Note that although submission of your assignment and the return of feedback will use Gradescope, your solution will not be autograded as the Gradescope autograder does not support OpenCL. Therefore you will not receive any information about the functionality of your submission until it has been manually assessed by staff.

5. **Submission requirements**

   It is expected that you will only modify the files `cwk3.c` and `cwk3.cl`, in which case these should be uploaded to the submission portal on Gradescope. As explained above, there is no autograder for this assignment.

   If you add any files, update the makefile if necessary, but do not change the executable name. Do not use subdirectories - keep all files in a flat directory - and **do not alter the file `helper_cwk.h` or remove calls to the routines it contains**, as it will be replaced with a modified version for the assessment.

6. **Academic misconduct and plagiarism**

   Academic integrity means engaging in good academic practice. This involves essential academic skills, such as keeping track of where you find ideas and information and referencing these accurately in your work.

   By submitting this assignment, you are confirming that the work is a true expression of your own work and ideas and that you have given credit to others where their work has contributed to yours.

   If you use material from outside the module material available on Minerva, include reference(s) in your comments.

There is a three-tier traffic light categorisation for using Gen AI in assessments. This assessment is **amber** category: AI tools can be used in an assistive role. Use comments in your code to declare any use of generative AI, making clear what tool was used and to what extent.

Code similarity tools will also be used to check for collusion.

7. **Assessment/marking criteria**

   There are 15 marks in total.

6 marks	:	Allocation and management of device memory.
6 marks	:	Parallel functionality of matrix transpose for matrices of various sizes.
3 marks	:	Kernel code, management, and execution.